REST API Design in Spring

7 min read · Updated 28 Jul 2026 · Topic 60 of 65

Bookmark · Mark done

Why it matters

Almost every Java backend role involves building HTTP APIs, and API design is where inexperience shows fastest — a 200 response containing an error, an endpoint named `/getUserById`, an entity returned straight from the controller. None of those break immediately, and all of them are expensive later, because a published API is a contract you cannot quietly change.

The concept, simply

REST models your system as **resources** identified by URLs, manipulated with the standard HTTP verbs. The nouns go in the path; the verb says what you are doing.

Verb	Purpose	Safe?	Idempotent?
`GET`	Read	Yes	Yes
`POST`	Create, or a non-CRUD action	No	**No**
`PUT`	Replace the whole resource	No	Yes
`PATCH`	Partial update	No	Not necessarily
`DELETE`	Remove	No	Yes

Idempotent means repeating the request has the same effect as making it once — which matters practically, because it decides whether a client may safely retry after a

timeout. `PUT` and `DELETE` can be retried; `POST` cannot, which is why creation endpoints need an idempotency key if duplicates would be harmful.

How it works

Status codes carry meaning and should be used precisely:

- **200** OK · **201** Created (with a `Location` header) · **204** No Content (a successful `DELETE`)
- **400** malformed or invalid · **401** not authenticated · **403** authenticated but not allowed · **404** not found · **409** conflict · **422** semantically invalid
- **500** our fault · **503** temporarily unavailable

The 401-versus-403 distinction is asked constantly: **401 means "I do not know who you are", 403 means "I know who you are and you may not do this"**.

Code examples

JAVA

Copy

```java
@RestController
@RequestMapping("/api/v1/orders")
public class OrderController {

    @GetMapping("/{id}")
    public OrderResponse get(@PathVariable String id) { … }        // 200

    @PostMapping
    public ResponseEntity<OrderResponse> create(@Valid @RequestBody Create
        OrderResponse created = service.create(req);
        return ResponseEntity
                .created(URI.create("/api/v1/orders/" + created.id()))   /
                .body(created);
    }

    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)                          // 204
    public void delete(@PathVariable String id) { service.delete(id); }
}
```

```
// X Verbs in the path, and a 200 that actually failed:
//   GET /getOrderById?id=42  → { "status": "error", "code": 404 }
//   Clients cannot use HTTP semantics, caches cannot work, and every
//   consumer has to parse a body to discover whether the call succeeded.
```

Never expose entities — the single most consequential rule here:

JAVA Copy

```
// X Returning the entity leaks the schema, exposes fields you never meant
//   publish (password hashes, internal flags), triggers lazy loading duri
//   serialisation (Chapter 12's N+1 in the web layer), and welds your API
//   contract to your database design.
@GetMapping("/{id}") public Order get(@PathVariable Long id) { … }

// ✓ A record DTO: explicit, minimal, and free to evolve separately.
public record OrderResponse(String id, String reference, OrderStatus statu
                            BigDecimal total, Instant createdAt) {
    public static OrderResponse from(Order o) {
        return new OrderResponse(o.getId().toString(), o.getReference(),
                                 o.getStatus(), o.getTotal(), o.getCreated
    }
}
```

Pagination and versioning, the two things people leave until it is too late:

JAVA Copy

```
// ✓ Always paginate a collection endpoint. An unbounded list works with 5
//   rows in development and takes the service down at 500,000.
@GetMapping
public Page<OrderResponse> list(@PageableDefault(size = 20) Pageable pagea
    return service.findAll(pageable).map(OrderResponse::from);
}
// ⚠ Cap the page size server-side — a client can otherwise request
//   ?size=1000000 and defeat the point entirely.

// ✓ Version from day one. Adding /v1 later means rewriting every client;
//   adding /v2 alongside /v1 costs nothing.
@RequestMapping("/api/v1/orders")
```

```
// fields. REMOVING or renaming one, or changing a type, breaks them — tha
// needs a new version, or a deprecation window with both live.
```

Under the hood

Idempotency in practice is more than trivia. A client that times out on a `POST` genuinely does not know whether the order was created. The standard answer is an idempotency key: the client sends a unique header, the server records it with the result, and a repeat of the same key returns the original response instead of creating a second order. Every payment API works this way, and it is a strong thing to raise unprompted.

The `Content-Type` and `Accept` headers drive Spring's message converters. A request whose type does not match produces 415 Unsupported Media Type, and an `Accept` the server cannot produce gives 406 — both frequently misdiagnosed as application bugs when they are content negotiation working as designed.

Consistency beats theoretical purity. Strict REST — HATEOAS, hypermedia controls — is rarely implemented fully, and that is fine. What actually matters to the teams consuming your API is that resources are nouns, verbs mean what HTTP says they mean, status codes are honest, errors have a stable shape, and collections are paginated. An API that is predictable is more valuable than one that is doctrinaire.

Common mistakes

- **Verbs in URLs.** `/getUser`, `/createOrder` — the HTTP method is the verb.
- **200 with an error body.** Breaks every client, cache and monitor that reads status codes.
- **Returning entities.** Leaks the schema, triggers lazy loading, welds API to database.
- **Unpaginated collections.** Fine in development, fatal in production.
- **No version in the path.** Adding one later breaks every client at once.
- **Confusing 401 and 403.** Unknown identity versus insufficient permission.

- **Leaking exception messages.** Stack traces and SQL in responses are an information disclosure.
- **Non-idempotent creation with no key.** A retried timeout creates duplicates.

Interview lens

Q: "What makes an API RESTful?"

A: Resources identified by URLs, manipulated with the standard HTTP methods, with the semantics of those methods respected — `GET` safe, `PUT` and `DELETE` idempotent — and honest status codes. In practice I care most about consistency: nouns in paths, correct verbs, accurate statuses, a stable error shape and paginated collections. Full HATEOAS is rarely worth it; predictability always is.

Q: "Which methods are idempotent, and why does it matter?"

A: `GET`, `PUT` and `DELETE` are; `POST` and generally `PATCH` are not. It matters because it decides whether a client can safely retry after a timeout. Since a create is a `POST`, a timed-out client cannot know whether it succeeded — which is why creation endpoints that must not duplicate take an idempotency key, storing the result against it so a repeat returns the original response.

Q: "401 or 403?"

A: 401 Unauthorized means the request is not authenticated — no credentials, or invalid ones — and the client should authenticate and retry. 403 Forbidden means we know who you are and you are not permitted; retrying with the same identity will not help. The names are historically confusing: 401 is really "unauthenticated".

Q: "Should you return entities from controllers?"

A: No. It exposes your database schema as your public contract, so a column rename becomes a breaking API change; it can leak fields you never intended to publish; and serialisation walks lazy associations, producing N+1 queries or

`LazyInitializationException` in the web layer. A record DTO built explicitly from the entity solves all three.

Q: "How do you version an API?"

A: A version segment in the path, from the first release — `/api/v1/`. Header or media-type versioning is arguably purer but harder to test and debug. The important part is deciding what counts as breaking: adding a field is safe if clients ignore unknown ones, while removing, renaming or retyping is not, and needs a new version running alongside the old with a deprecation window.

Q: "How would you handle a client requesting a huge page size?"

A: Cap it server-side. Pagination that a client can defeat with `?size=1000000` is not pagination — it is an easy denial of service and a slow query for the database. I set a default and a hard maximum in configuration and clamp anything above it, returning the capped page rather than an error.

Cheat-card

- **Nouns in paths, verbs in the method.** No `/getUser`.
- Idempotent: `GET` `PUT` `DELETE`. **Not** `POST` → idempotency key for retries.
- 201 + `Location` on create · 204 on delete · **401 unauthenticated · 403 forbidden** · 409 conflict.
- **Never return entities** — record DTOs. Never leak exception messages.
- **Always paginate**, and cap the page size server-side.
- **Version from day one** (`/api/v1`). Adding a field is safe; removing/renaming is not.
- 415 = wrong `Content-Type`, 406 = unsatisfiable `Accept` — content negotiation, not bugs.

QUICK CHECK — no marks, just for you.

- [ ] All of them — HTTP requires idempotency
- [ ] Only GET, because it does not modify anything
- [ ] GET, PUT and DELETE — it decides whether a client can safely RETRY after a timeout, which is why POST creation endpoints need an idempotency key
- [ ] POST and PATCH, because they are the write methods

Check answer

2. What is the difference between 401 and 403?

- [ ] 401 means the resource does not exist; 403 means it is deleted
- [ ] 401 is for APIs and 403 is for web pages
- [ ] They are interchangeable
- [ ] 401 means not authenticated — authenticate and retry; 403 means authenticated but not permitted — retrying with the same identity will not help

Check answer

3. Why should a controller never return a JPA entity directly?

- [ ] It exposes the database schema as your public contract, can leak fields you never meant to publish, and serialisation walks lazy associations — producing N+1 or LazyInitializationException in the web layer
- [ ] Because it is slower than a DTO
- [ ] Because entities are not thread-safe

Because entities cannot be serialised to JSON

Check answer

Previous
@Transactional: Propagation, Isolation and Traps

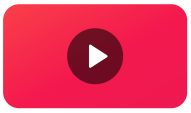
Next
Spring Security and JWT

LEARN MORE

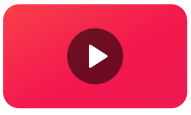
ARTICLES & DOCS

- REST clients and controllers — Spring Framework reference
  docs.spring.io
- HTTP semantics — RFC 9110
  www.rfc-editor.org
- Problem details — RFC 9457
  www.rfc-editor.org

VIDEOS



"rest api design" — video explainers
YouTube



rest api design interview questions
YouTube

Hand-picked references for this lesson.